IoT MINI PROJECT · PHASE 2: RESULTS

Addressing Class Imbalance in CICIoT2023-Based Intrusion Detection via SMOTE and Feature Selection



Dhruv Sandilya · 231CS122 · NIT Karnataka

Base paper: E. C. P. Neto et al., “CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment,” Sensors 23(13):5941, 2023

Three questions, three answers

RQ1 · Reproduce

0.995

8-class accuracy (paper: 0.994). Same blind spot on web and brute-force attacks.

RQ2 · Improve

+0.066

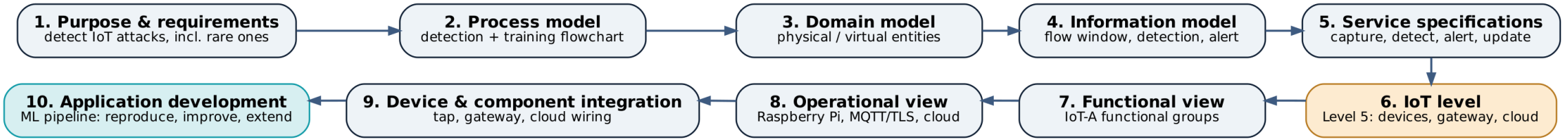
Macro-F1 of the Random Forest (0.785 → 0.851). Every method beats the baseline, $p < 0.05$.

RQ3 · Extend

20 of 46

features keep the full macro-F1 (0.856), 22% faster, 28% smaller.

Designed with the 10-step IoT design methodology



Steps 1–5 · What and how

Purpose: catch attacks, incl. rare ones, at the gateway. Process flowchart, domain and information models, eight services.

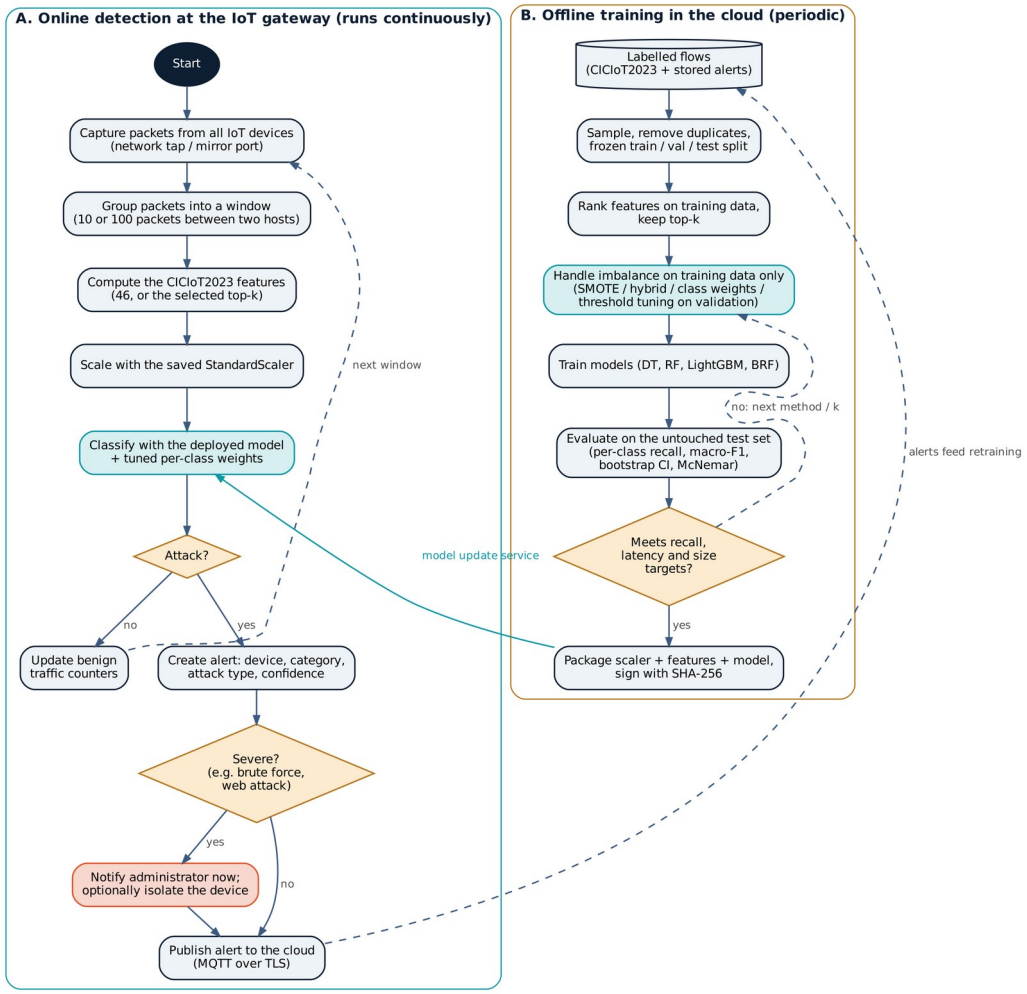
Steps 6–9 · Where it runs

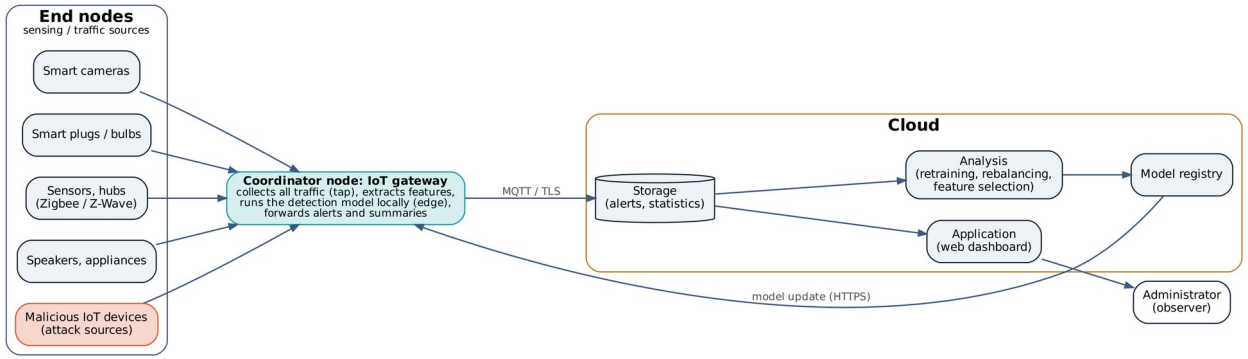
Level 5: IoT devices as end nodes, the gateway as coordinator running detection, the cloud for storage, training and the dashboard.

Step 10 · Application

The detection model and the ML pipeline that builds it: reproduce, improve, extend. The rest of this talk.

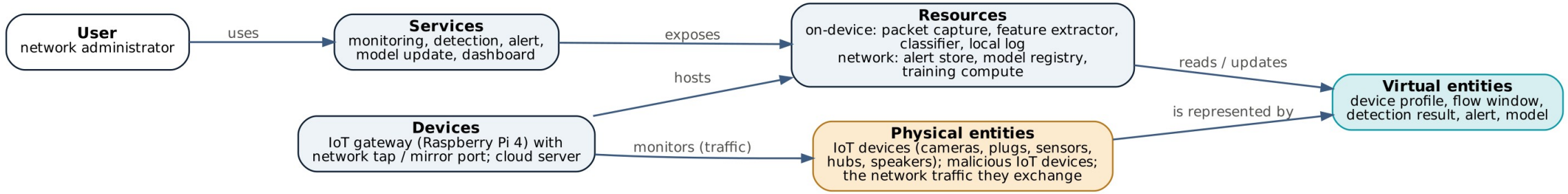
Process model and deployment level

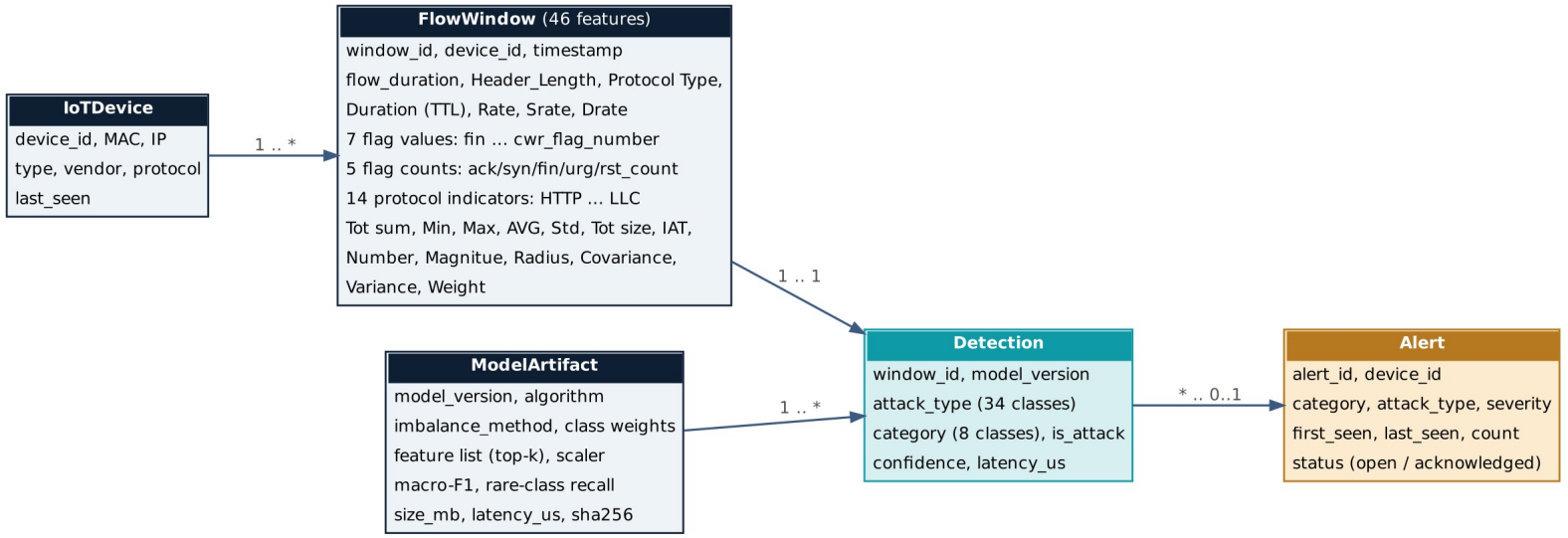




Level 5, with detection at the coordinator: the gateway sees a copy of all traffic and classifies it in microseconds; only alerts go to the cloud, and models come back from it.

Domain and information models

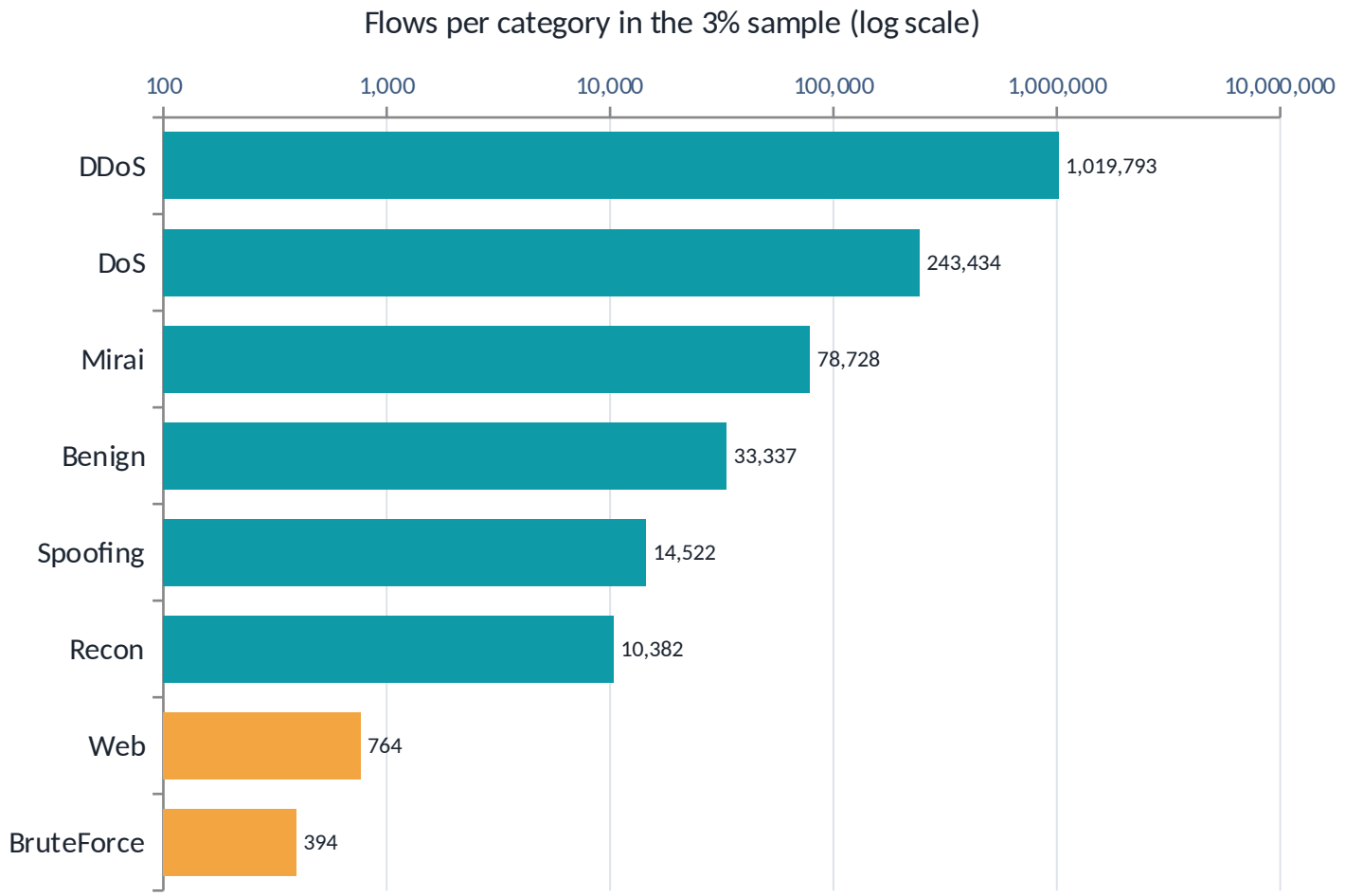




46 features per window

Each FlowWindow summarises 10 or 100 packets between two hosts: rates, flags, protocol indicators and packet-size statistics, as in the CICIoT2023 CSV release.

The data: 1.4 million flows, 2,588 : 1 imbalance



1.40 M

flows from all 169 files (3.0% of 46.7 M), class ratios kept

166,898

exact duplicate rows removed before splitting

15.9%

of test rows would have an exact copy in training without de-duplication

Reproduction: the paper's accuracy, reproduced

Random Forest	Accuracy, paper	Accuracy, ours	Macro-F1, paper	Macro-F1, ours
2 classes	0.997	0.997	0.965	0.970
8 classes	0.994	0.995	0.719	0.785
34 classes	0.992	0.993	0.714	0.751

Accuracy agrees within 0.002 at every granularity. Our macro-F1 is higher, likely from the de-duplicated stratified split and a different training procedure.



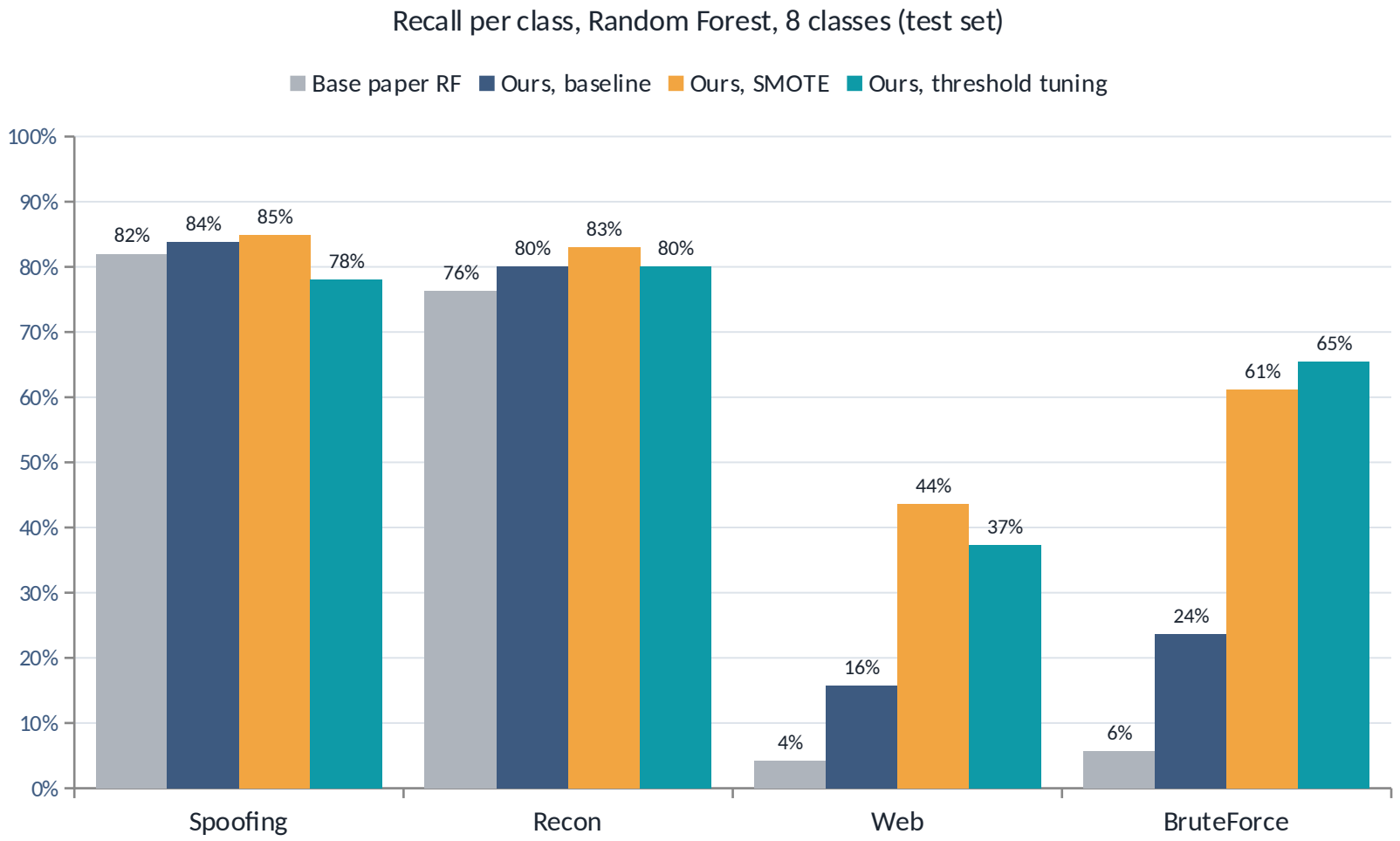
The paper's precision and recall are swapped

From the paper's own RF confusion matrix (Table 8): recall 0.706, precision 0.917.

Table 6 prints them the other way round (0.910 / 0.705).

Cause: the example code calls recall_score(y_pred, y_test); scikit-learn expects (y_true, y_pred).

The blind spot, and how the fixes close it



6% → 65%

brute-force recall: the paper's RF vs our threshold-tuned RF

4% → 44%

web-attack recall: the paper's RF vs our SMOTE RF

Every method beats the baseline (Random Forest)

Method	Macro-F1	Rare-class recall	Web recall	BruteForce recall	Model size	Bootstrap p
Baseline	0.785	0.398	0.157	0.236	169 MB	–
SMOTE	0.837	0.626	0.436	0.612	468 MB	< 0.001
SMOTE + undersampling	0.837	0.626	0.429	0.612	449 MB	< 0.001
Class weights	0.841	0.523	0.314	0.477	227 MB	< 0.001
Borderline-SMOTE	0.811	0.481	0.355	0.262	487 MB	0.013
Threshold tuning	0.851	0.609	0.373	0.654	169 MB	< 0.001

Threshold tuning gives the best macro-F1 at no extra training data or size. **SMOTE** finds the most rare attacks but cuts web precision (0.901 → 0.443) and makes the forest 2.8× larger.

Significant, not noise

+0.066

macro-F1 with threshold tuning; 95% CI [+0.042, +0.090] across seeds

+0.228

rare-class recall with SMOTE (Recon, Web, BruteForce averaged)

5 of 5

methods significant (bootstrap $p < 0.05$) for every seed

How it was tested: each method against the baseline trained with the same seed, on the same 246,892 test rows: 1,000 paired bootstrap resamples and McNemar's exact test.

Why McNemar can disagree for threshold tuning: it swaps a few majority-class errors for many rare-class hits, so the count of correct rows barely moves while the balance across classes improves.

Four models: quality against size



Decision Tree

Class weights: macro-F1 0.841 in 1.1 MB, 0.11 µs per flow.

Balanced RF

Highest rare-class recall (0.771) but low precision: macro-F1 0.705.

LightGBM

Unstable without rebalancing (0.503 ± 0.13); rebalancing fixes it (0.833).

34 classes: the hardest case

4

attack types the baseline forest never detects (Backdoor, SQL injection, ping sweep, uploading)

0.751 → 0.807

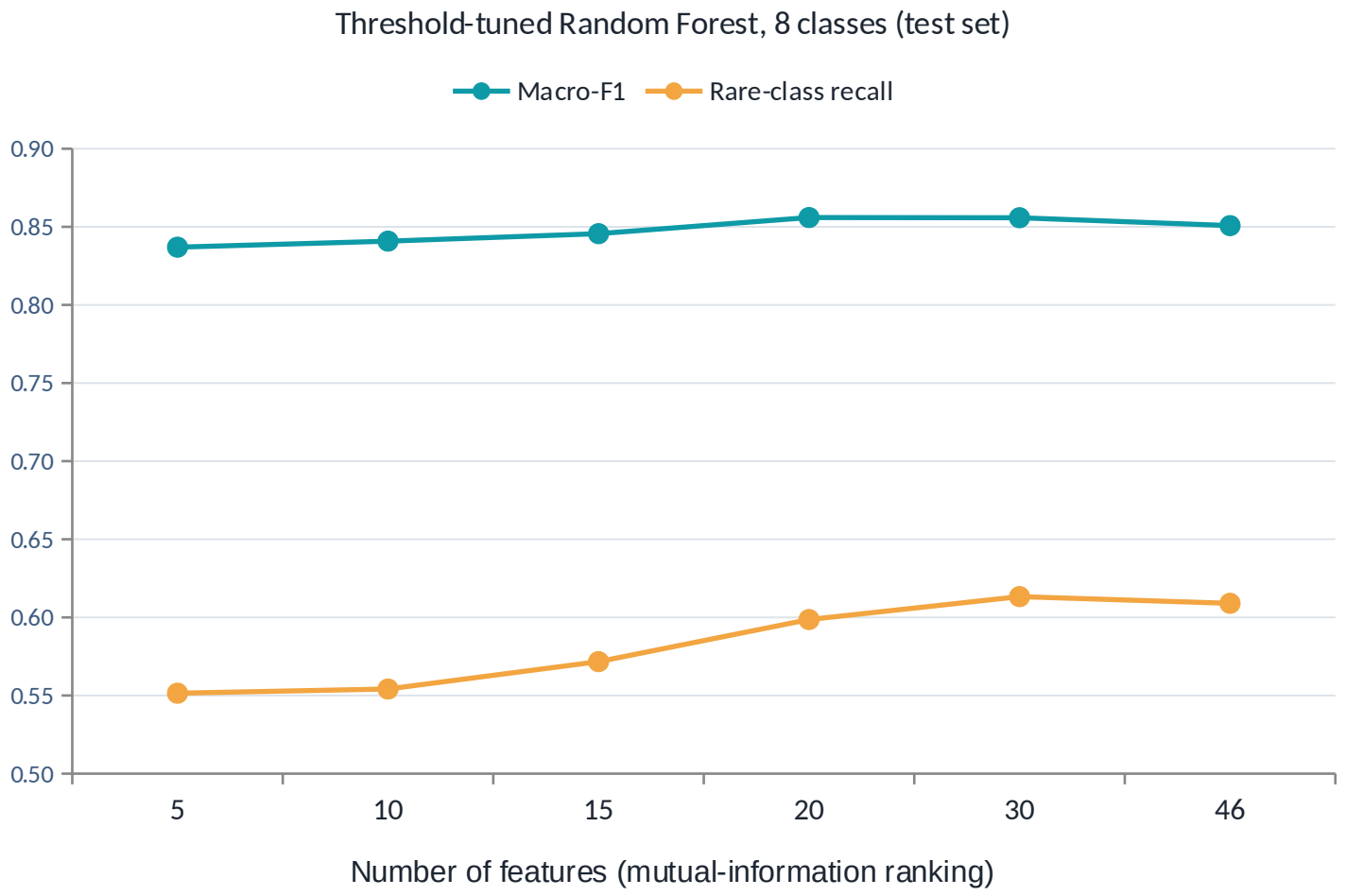
macro-F1 with SMOTE, the largest gain here ($p < 0.001$)

0.526 → 0.650

rare-class recall with SMOTE

With 34 classes the rarest attack types have only 8–33 test rows, so per-type recall is noisy; **the 8-class task is the main result.**

Extension: 20 of 46 features is enough



−22%

prediction time per flow (4.02 → 3.12 µs)

−28%

model size (169 → 121 MB)

0.856

macro-F1 with 20 features (0.851 with all 46)

Which features matter

Rank	Mutual information	RF importance	RFE
1	IAT	IAT	IAT
2	Tot size	Min	Min
3	Max	flow_duration	Magnitue
4	Magnitue	Protocol Type	flow_duration
5	AVG	Tot size	Tot size
6	Tot sum	Magnitue	Max
7	Min	Max	Tot sum
8	Header_Length	AVG	rst_count

IAT ranks first in all three

Packet inter-arrival time separates the floods from everything else; packet-size statistics and flow duration follow.

Rankings agree

Filter (MI), embedded (RF) and wrapper (RFE) methods pick largely the same top features, so the selection is robust.

What to run on the gateway

Most accurate

Random Forest + threshold tuning, 20 features

- Macro-F1 0.856
- 3.1 µs per flow
- 121 MB

Most deployable

Decision Tree + class weights

- Macro-F1 0.841
- 0.11 µs per flow
- 1.1 MB (111× smaller)

Latency is batch prediction on a 4-core Kaggle CPU; on a Raspberry Pi, packet capture and feature extraction will dominate.

Limitations and future work

Limitations

- A 3% sample; the rarest 34-class types have few test rows
- One dataset from one testbed; no replay attacks in CICIoT2023
- Latency measured on a Kaggle CPU, without feature extraction
- LightGBM run with default settings
- Gateway, MQTT and dashboard designed, not deployed

Future work

- Deploy the selected model on a Raspberry Pi with live capture
- Test transfer to CIC IoT-DIAD 2024 and the Dalhousie IoT 2024 dataset
- Tune LightGBM for imbalance
- Two-stage detector: floods first, then a specialist for the rest

Conclusion

A Level 5 gateway IDS, designed with the 10-step methodology, whose detector now finds the rare attacks the benchmark missed.

Reproduced

accuracy within 0.002 of the paper; its precision/recall swap found

Improved

macro-F1 0.785 → 0.851; rare recall up to 0.626; $p < 0.05$

Extended

20 features: same quality, 22% faster, 28% smaller

Thank you · Questions?